Quest 2: The Axeperiment

Lord of the Wings · ranking Gimlock's alloys from hardest to softest

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: given head-to-head hardness results between alloys, find the order from hardest to softest, read it as a number, and add these numbers up.
Input: one line per group of alloys, such as `4<3 2<5 3>1 …`
Output: the checksum, the sum of the order-numbers.
What changes between parts: how many comparison results are missing (none, exactly one, or many).

1. The story

The party stops at the Forge Mountain to recruit the dwarven blacksmith **Gimlock Ironstack**. He will only leave once his research on the hardness of new metal alloys is finished. His method is to hit two axe blades together and see which one gives way. You help him make sense of his notes.

2. Understanding the question

Reading one comparison

Each token compares two alloys. The pointy side of the symbol points at the **softer** one:

Token	Means	Rewritten as harder>softer
`3>5`	alloy 3 is harder than alloy 5	`3>5`
`4<3`	alloy 4 is softer than alloy 3	`3>4`

From comparisons to an order

Each line is one group. Arrange its alloys from hardest to softest and read the alloy numbers as one number. Here is a small 4-alloy example (not from the puzzle):

```
1<2 3>1 2<3 4<1 4<2 4<3
rewritten:  2>1  3>1  3>2  1>4  2>4  3>4
```

Alloy 3 beats everyone, then 2, then 1, then 4, so the order is **3 2 1 4** and this group's number is **3214**. The **checksum** is the sum of these numbers over all lines.

> **Key insight used in every part:** hardness is a strict order with no ties and no cycles. So once you know the result for **every pair**, the hardest alloy beats all n – 1 others, the second hardest beats n – 2, and so on down to the softest, which beats none. **Counting how many alloys each one beats gives its position directly**, and sorting by that count gives the order. In the 4-alloy example the win counts are 3:3, 2:2, 1:1, 4:0.

3. Shared code: parsing and turning win counts into a number

All three parts share the same first step (parse and rewrite) and last step (count wins, sort, join). Here they are from Part I:

```
10  const getAnswer = (input: string) => {
11    const pairs = input
12      .split(" ")
13      .map((v) =>
14        v.includes("<") ? [...v].reverse().join("").replace("<", ">") : v,
15      );
16
17    const wins = Array(5).fill(0);
18
19    pairs.forEach((p) => wins[Number(p[0]) - 1]++);
20
21    const nums: {
22      [key: number]: number;
23    } = {};
24
25    wins.forEach((w, index) => {
26      nums[Number(index + 1)] = w;
27    });
28
29    const answer = Number(
30      Object.entries(nums)
31        .sort(([, a], [, b]) => b - a)
32        .map(([v]) => v)
33        .join(""),
34    );
35
36    return answer;
37  };
```

q2/part1.ts, lines 10–37

Lines	What happens
11–12	Splits the line on spaces into tokens: `["4<3", "2<5", …]`.
13–15	Rewrites every `<` token as `harder>softer`. The trick: reverse the characters, then swap the symbol. `"4<3"` → `["4","<","3"]` reversed → `"3<4"` → replace `<` with `>` → `"3>4"`. Tokens with `>` are left as they are.
17	One win counter per alloy. `wins[0]` is alloy 1, `wins[4]` is alloy 5. Part I's real input has 5 alloys per group, so 5 is hard-coded.
19	In each token `"a>b"`, the first character `p[0]` is the harder alloy, so that alloy's counter goes up by one. This works because alloy ids are single digits (1–9).
21–27	Copies the counters into an object `{{1: wins, 2: wins, …}}` keyed by alloy number.
29–34	`Object.entries` gives `[alloy, wins]` pairs. Sorting by wins, highest first, gives the order. `.map(([v]) => v)` keeps just the alloy numbers, `.join("")` glues them into a string such as `"35124"`, and `Number(...)` turns that into a number.
39–45	Adds up the numbers for every line and prints the checksum.

4. Part I: every pair is compared

The question

Each group contains the result of **every possible pair**. With 5 alloys that is 5 × 4 / 2 = 10 comparisons per line.

Step-by-step solution

1. Take one line (one group) and split it on spaces into comparison tokens.
2. Rewrite every token as `harder>softer`: `4<3` becomes `3>4`.
3. For each token, add 1 to the win count of the alloy on the left of `>`.
4. Sort the alloys by win count, most wins first. That is hardest to softest.
5. Glue the alloy numbers together into one number, such as 35124.
6. Do this for every line and add up the numbers. That is the checksum.

Worked example: first sample line

```
4<3 2<5 3>1 1>2 1<5 4<1 3>2 4<2 3>5 4<5
rewritten (line 13–15):
3>4 5>2 3>1 1>2 5>1 1>4 3>2 2>4 3>5 5>4
```

Count the winners (line 19), looking at the first character of each rewritten token:

Alloy	1	2	3	4	5
wins	2	1	4	0	3

Sorted by wins: 3 (4), 5 (3), 1 (2), 2 (1), 4 (0) → `"35124"` → **35124**.

Line	Order	Number
`4<3 2<5 3>1 1>2 1<5 4<1 3>2 4<2 3>5 4<5`	3 5 1 2 4	35124
`4>1 2>1 3>2 1<5 4>5 2<5 3>1 4>3 2<4 5<3`	4 3 5 2 1	43521
`5>1 5>2 4>1 2<4 3>2 3<4 1>2 3>5 1<3 4>5`	4 3 5 1 2	43512
Checksum		**122157**

5. Part II: exactly one comparison is missing

The question

Each group in the second notebook is missing **exactly one** comparison. Without it, two alloys would have the same win count and their order would be unclear. Frodoff's observation is that hardness is **transitive**:

```
if  a > c  and  c > b  then  a > b
```

So you can recover the missing result through some third alloy `c` and then count wins as before. The real input has 9 alloys per group, so the numbers are 9 digits long.

Worked example: first sample line

```
2<5 3>1 1>2 1<5 4<1 3>2 4<2 3>5 4<5        ← only 9 tokens
rewritten:  5>2 3>1 1>2 5>1 1>4 3>2 2>4 3>5 5>4
```

Alloys 3 and 4 were never compared. But `3>5` and `5>4`, so **3>4**. Also `3>1` and `1>4` lead to the same result. Now every pair is known and the order is 3 5 1 2 4 again. The sample is the Part I sample with one comparison removed from each line, so the checksum is still **122157**.

Step-by-step solution

1. Parse and rewrite the tokens as in Part I. One pair is missing.
2. For every pair of alloys (a, b), look for a third alloy c with `a>c` and `c>b`. If you find one, add `a>b`. Check the other direction (`b>c>a`) too.
3. The missing pair is now filled in.
4. Count wins, sort, glue the numbers together and add them up, as in Part I.

My solution: the new part

```ts
const MAX = isSample ? 5 : 9;

const getAnswer = (input: string) => {
  const pairs = input
    .split(" ")
    .map((v) =>
      v.includes("<") ? [...v].reverse().join("").replace("<", ">") : v,
    );

  for (let a = 1; a <= MAX; a++) {
    for (let b = a + 1; b <= MAX; b++) {
      for (let c = 1; c <= MAX; c++) {
        if (c === a || c === b) continue;

        if (pairs.includes(`${a}>${c}`) && pairs.includes(`${c}>${b}`))
          pairs.push(`${a}>${b}`);

        if (pairs.includes(`${b}>${c}`) && pairs.includes(`${c}>${a}`))
          pairs.push(`${b}>${a}`);
      }
    }
  }

  const wins = Array(MAX).fill(0);

  pairs.forEach((p) => wins[Number(p[0]) - 1]++);
```

q2/part2.ts, lines 10–35 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
10	The number of alloys: 5 in the sample, 9 in the real input.
19–20	Loops over every unordered pair `(a, b)` with `a < b`.
21–22	Tries every other alloy `c` as the middle step.
24–25	If `a>c` and `c>b` are both known, adds `a>b`.
27–28	Checks the other direction too: if `b>c` and `c>a`, adds `b>a`.
33–35	Counts wins over the extended list, then sorts and joins as before.

A subtle detail worth knowing: line 25 adds `a>b` **every time** it finds a middle alloy, even if `a>b` is already in the list. So `pairs` ends up with duplicates, and the win counts are inflated. For the first sample line, the instrumented run gives 19 entries instead of 10:

```
correct wins (alloys 1..5):  [2, 1, 4, 0, 3]
wins from line 35:           [3, 1, 9, 0, 6]
```

The answer is still right because of how the duplicates are spread. An alloy with k alloys below it gets extra entries for the pairs it can reach through a middle alloy. That bonus **grows with k**, so stronger alloys always get at least as many extras as weaker ones and the **order is preserved**. It works, but by luck rather than design. Part III's version uses a `Set`, which avoids this.

6. Part III: many comparisons missing

The question

The early notes are missing **many** results, for example:

```
5>1 3<4 1>2 3>5            ← only 4 of the 10 pairs
```

A single middle alloy is no longer enough. To know that 4 is harder than 2, you need the chain `4 > 3 > 5 > 1 > 2`. You need every result that follows from **chains of any length**, which is called the **transitive closure**. The puzzle guarantees that this is enough to fix the complete order.

Step-by-step solution

1. Parse and rewrite the tokens. Store them in a **Set** of known pairs.
2. For each alloy c (1, 2, 3, … as the **outer** loop): for every a with `a>c` known, and every b with `c>b` known, add `a>b` to the set.
3. After the last c, every pair that follows from any chain is in the set (the transitive closure).
4. Count wins, sort, glue the numbers together and add them up, as in Part I.

My solution: the new part

```
19  const known = new Set(pairs);
20
21  for (let c = 1; c <= MAX; c++) {
22    for (let a = 1; a <= MAX; a++) {
23      if (!known.has(`${a}>${c}`)) continue;
24
25      for (let b = 1; b <= MAX; b++) {
26        if (known.has(`${c}>${b}`)) known.add(`${a}>${b}`);
27      }
28    }
29  }
30
31  const wins = Array(MAX).fill(0);
32
33  known.forEach((p) => wins[Number(p[0]) - 1]++);
```

q2/part3.ts, lines 19–33 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
19	Puts the known results in a `Set`. This gives fast `has` lookups and stops the same pair from being stored twice.
21	**The middle alloy `c` is in the outer loop.** This is the Floyd–Warshall order, and it's what makes long chains work.
22–23	For each alloy `a` that is harder than `c` …
25–26	…and each `b` that `c` is harder than, records `a>b`.
33	Counts wins from the closed set. Each pair now appears exactly once, so the counts are the true positions.

Why the middle alloy has to be in the outer loop

After the outer loop has handled `c = 1 … k`, the set contains `a>b` whenever there is a chain from a to b whose **middle alloys are all ≤ k**. When k reaches the last alloy, every chain is covered. If `c` were in an inner loop, as in Part II, a result found late in the loops couldn't be used to find earlier ones, so long chains could be missed.

Trace: third sample line `5>1 3<4 1>2 3>5`

Start: `{5>1, 4>3, 1>2, 3>5}`

c	a with a>c	b with c>b	Added
1	5	2	`5>2`
2	1, 5	(none)	-
3	4	5	`4>5`
4	(none)	3	-
5	3, 4	1, 2	`3>1 3>2 4>1 4>2`

The set now has all 10 pairs. Wins are 1:1, 2:0, 3:3, 4:4, 5:2, so the order is 4 3 5 1 2 → **43512**. When c = 5, the loop used `3>5` and `5>2`; `5>2` had only just been added when c = 1. That is the chain-building at work.

Line	Number
`3>1 1>2 1<5 4<2 3>5`	35124
`2>1 2<5 3>1 4>3 2<4 5<3`	43521
`5>1 3<4 1>2 3>5`	43512
Checksum	**122157**

7. Summary

Part	Missing	How the gaps are filled	Code
I	none	nothing to fill; just count wins	array, count, sort
II	exactly 1	one middle alloy is enough	triple loop, c innermost (duplicates are harmless)
III	many	full transitive closure	Set, triple loop, c outermost (Floyd–Warshall)

> Another way to see Part III: draw an arrow a → b for "a is harder than b". The answer is the unique **topological order** of that graph, so Kahn's algorithm would also solve it.